

Hotel Room Reservation System – Project Report

Martim Alves 1240690, Ricardo Monteiro 1240842, Rodrigo Martins 1241122, Tiago Caetano 1241945

Abstract: This report documents the final stage of the development of the Hotel Room Reservation System. It reports needed changes, as well as some changes by preference, made to the previous iteration and every new aspect of the code for the system. There is a well-organized and updated version of the deployment of every part of the system, as well as a fluid connection between them. The report presents elements to provide a complete view of the system's coding progress.

Keywords: Project, Final Iteration, Controller, Model, Views

1. Introduction

The third iteration of the Hotel Reservation System marks a significant milestone in the project's development. With the foundation laid with the previous iterations of the project, this iteration focuses on the transition from design to implementation and testing. The code for the system has now been developed and will be presented in this report, with the realization of the project architecture previously documented.

With ongoing progress, changes needed to be made, along with facultative changes that the group thought would not affect the system's functionality in negative ways. Those changes will also be addressed. Along with changes came the refinement of functionalities, and especially the interface. With the already intuitive menu, we also made it so every part of the program would have the same ease of use.

This report includes a detailed view of the project's functionalities and everything that makes the system work, by having everything work in harmony through the MVC (Model, View, Controller) type of architecture. This is the last project report, with the documentation of the completed project.

2. Changes

In the second iteration of our design, the user interface remained relatively simple and included functionality that we later determined was unnecessary or unlikely to be usable. One example of this is the Language Settings menu. Although initially considered, we decided not to implement it. This decision was based on the fact that our entire codebase is written in English—a universal language widely used in software development—making the inclusion of a language selector redundant and unnecessarily laborious.

In addition to removing non-essential menus, we also expanded the interface by introducing submenus. This was done to ensure that each main menu option logically led to a dedicated submenu for that option, such as in the login process. However, we also recognize that our ViewModel diagram remained intentionally simple. We know that the design choice was initially made to increase clarity and ease of understanding for all stakeholders involved in the project without overly in-depth or superficial details.

3. Project Architecture

The project is structured following a modular and layered approach, inspired by the Model-View-Controller (MVC) design pattern. This structure enables a clear separation of concerns between the application's data representation, user interface, and control logic, allowing for better organization, scalability, and maintainability.

The project is primarily divided into two main directories: `headers/` and `sources/`. Each of these directories is further divided into subfolders that categorize the project's components by functionality.

The `headers/` directory contains all the declarations of classes, enumerations, and utility functions used across the system. It is organized as:

- `headers/model/`: This folder defines the core data models that represent the main entities of the system. These include classes such as `Hotel`, `Room`, `Client`, `Reservation`, `Service`, `Activity`, `Comment`, and related containers like `RoomContainer`, `ActivityContainer`, and `ReservationContainer`. These models are responsible for encapsulating the data attributes and basic behavior of the business objects.
- `headers/views/`: This folder includes the classes that manage the user interface through textual menus and input prompts. These views provide the interaction layer for the user and are responsible for operations such as displaying hotel and room information, registering activities, processing payments, writing reviews, and managing accounts.
- `headers/controller/`: This folder contains the `Controller` class, which serves as the central component that commands the application logic. It connects actions between the views and models, manages the flow of the program, and ensures correct execution of operations such as room booking, reservation management, service and activity handling, and user authentication.
- `headers/data/`: This directory includes enumerations and supportive definitions used throughout the project. Examples include the `RoomType` and `ReservationStatus` enums, as well as the data seeding header that facilitates the creation of sample data for hotels and rooms during initialization.
- `headers/exceptions/`: This folder declares custom exception classes designed to handle specific types of runtime errors that may occur during execution, such as invalid login credentials, unavailable rooms or services, invalid coupons, and more. These exceptions provide meaningful feedback and help maintain robustness in the system.
- `headers/utils/`: This utility folder contains helper functions for common operations such as reading input from users, formatting data, and handling dates. It provides reusable code that supports the smooth functioning of the application.

The `sources/` directory contains the corresponding implementation files for each header. Its subfolders mirror the structure of the `headers/` directory and contain the actual code logic that defines how the declared classes and functions behave. For example:

- `sources/model/` implements the internal behavior of the data entities.
- `sources/views/` implements the logic behind the user interface components.
- `sources/controller/` provides the functionality of the main controller class.
- `sources/data/` initializes and generates example hotels and rooms.
- `sources/exceptions/` defines the messages and behavior of the exception classes.
- `sources/utls/` implements utility functions used throughout the system.

Each part of the architecture is responsible for a specific layer or aspect of the system, ensuring that the codebase is organized and responsibilities are clearly distributed across the different components.

4. Component Details

This section presents a detailed breakdown of the header (.h) and implementation (.cpp) files included in the project. Each file plays a specific role in defining the structure, logic, and functionality of the hotel reservation system.

4.1 Model Files

Located in `headers/model/` and `sources/model/`:

- `Hotel.h / Hotel.cpp`
Declares and implements the `Hotel` class, which encapsulates information about a hotel, including its rooms, services, activities, reservations, and comments.
- `Room.h / Room.cpp`
Defines the `Room` class, representing an individual hotel room with attributes such as number, price, type, and availability. Includes methods to access and modify these properties.
- `Client.h / Client.cpp`
Represents a system user with attributes like name, email, balance, and payment history. Implements methods for updating and retrieving client information.
- `Reservation.h / Reservation.cpp`
Handles reservation data, linking a client to a room for a specified period. Includes payment status and price calculation logic.

- Service.h / Service.cpp
Declares services offered by the hotel (e.g., spa, laundry), including price and availability.
- Activity.h / Activity.cpp
Represents hotel activities. Stores data about data like schedule, description, price, and participant limits.
- Comment.h / Comment.cpp
Defines user comments with rating and date. Used for feedback purposes.
- Payment.h / Payment.cpp
Manages payment records including amount, payment date, and unique identifiers.
- Cancellation.h / Cancellation.cpp
Stores cancellation details such as reason and date.
- Containers
 - RoomContainer.h / .cpp, ServiceContainer.h / .cpp, ActivityContainer.h / .cpp, and ReservationContainer.h / .cpp manage internal collections of the respective entities, providing methods for adding and retrieving elements.

4.2 View Files

Located in headers/views/ and sources/views/:

- MainView.h / MainView.cpp
Displays the main menu and routes users to the respective sections of the system.
- AccountView.h / AccountView.cpp
Provides interfaces for user account creation and login.
- SearchView.h / SearchView.cpp
Handles the search process for available rooms and manages reservations.
- RoomView.h / RoomView.cpp
Displays room details, lists of rooms, and messages for unavailable rooms.

- ServiceView.h / ServiceView.cpp
Allows users to view and request available services.
- ActivitiesView.h / ActivitiesView.cpp
Displays hotel activities and manages activity registration.
- PaymentView.h / PaymentView.cpp
Processes user payments and prints receipts.
- InformationView.h / InformationView.cpp
Presents detailed hotel and room information.
- ReviewView.h / ReviewView.cpp
Handles review submission, star ratings, and user feedback input.

4.3 Controller Files

Located in headers/controller/ and sources/controller/:

- Controller.h / Controller.cpp
Defines and implements the main control flow of the application. Coordinates the communication between the user interface and the model, tracks user sessions, and manages hotel operations, reservations, services, payments, and reviews.

4.4 Exception Files

Located in headers/exceptions/ and sources/exceptions/:

Each exception class (.h and .cpp) handles a specific runtime error case, providing custom error messages through what() implementations:

- Examples:
 - InvalidCredentialsException
 - RoomNotAvailableException
 - PaymentFailedException

- CouponInvalidException
- ActivityUnavailableException, etc.

These enhance error handling and provide meaningful user feedback.

4.5 Utility Files

Located in headers/utils/ and sources/utils/:

- Utils.h / Utils.cpp
Provides utility methods for user input, date management, and pausing the program, centralizing common logic used throughout the application.

4.6 Data Files

Located in headers/data/ and sources/data/:

- seed.h / seed.cpp
Declares and implements a function that generates a predefined list of fictional hotels and rooms for testing and demonstration purposes.

RoomType.h and ReservationStatus.h

Define enumerations used to categorize room types and reservation states, improving type safety and code readability.

5. Conclusion

This final report had as an objective the implementation of what were previously only structural architectures to a fully functional application. This project was a long-term evolution both in terms of learning as well as in project productivity, as with time and with each iteration, the group understood better the previous aspects of the work to be made, making each iteration easier to understand and making more sense than the previous one.

Naturally there were challenges that we needed to overcome, but we hope to have accomplished the objectives that were initially proposed.